MODUL 21 · BAB VI · ALGORITMA GRAF

Algoritma Prim untuk Minimum Spanning Tree

Menumbuhkan satu tree dari sebuah akar dengan selalu mengambil simpul terdekat yang belum masuk tree.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan langkah-langkah algoritma Prim serta arti atribut $v.key$ dan $v.\pi$.
- Menjalankan Prim pada graf contoh dan mencatat nilai key setiap simpul pada setiap iterasi.
- Menjelaskan mengapa simpul yang di-extract Prim selalu menambahkan safe edge.
- Menganalisis running time Prim dengan antrean prioritas berbentuk array dan binary heap.
- Membandingkan Prim dengan Kruskal.

PRASYARAT

- Modul 19: konsep MST, cut, dan light edge.
- Modul 16: adjacency-list graf berbobot.
- Modul 20 (disarankan): algoritma Kruskal sebagai pembanding.

BAB / MATERI ASAL

Materi 12. MINIMUM SPANNING TREE, algoritma Kruskal dan Prim (bagian Algoritma Prim)

POSISI DALAM URUTAN BELAJAR

Algoritma MST kedua. Strukturnya hampir sama dengan Dijkstra (Modul 24).

DAFTAR ISI

1. Tumbuh dari Satu Titik 1
2. Pseudocode Prim 2
3. Penelusuran Prim dengan Akar a 3
4. Kebenaran, Running Time, dan Perbandingan 5
5. Kesimpulan 7
- Kuis Pemahaman 8
- Kunci Jawaban dan Pembahasan 9
- Referensi 9

PENDAHULUAN

1. Tumbuh dari Satu Titik

Kruskal (Modul 20) membangun MST dari banyak potongan kecil yang lama-kelamaan menyatu. **Algoritma Prim** (Prim, 1957) memakai cara yang berbeda: mulai dari satu simpul akar r, lalu **terus menumbuhkan satu tree**. Di setiap langkah, Prim menambahkan simpul di luar tree yang dapat dihubungkan ke tree dengan busur **paling ringan**.

Bayangkan perusahaan kabel yang mulai dari kantor pusat. Setiap hari mereka menyambungkan satu kota baru: kota yang paling murah disambungkan ke jaringan yang sudah ada. Setelah semua kota tersambung, jaringan yang terbentuk adalah MST.

Untuk memilih "kota termurah berikutnya" dengan cepat, Prim menyimpan setiap simpul di luar tree beserta biaya termurah untuk menyambungkannya, dalam sebuah **antrean prioritas minimum**. Modul ini membahas pseudocode Prim dari bahan kuliah, penelusurannya pada graf contoh dengan akar a, dan analisis running time-nya.

PEMBAHASAN INTI

2. Pseudocode Prim

MST-PRIM(G, w, r)

```
1  for each u ∈ G.V
2      u.key = ∞
3      u.π = NIL
4  r.key = 0
5  Q = G.V
6  while Q ≠ ∅
7      u = EXTRACT-MIN(Q)
8      for each v ∈ G.Adj[u]
9          if v ∈ Q and w(u, v) < v.key
10             v.π = u
11             v.key = w(u, v)
```

Parameter masukan (G, w, r): G adalah graf tak berarah yang direpresentasikan sebagai adjacency-list, w adalah bobot setiap busur, dan r adalah simpul awal (akar).

Notasi	Arti
$u.key$	Nilai simpul u: bobot busur termurah yang menghubungkan u dengan tree saat ini. ∞ berarti belum ada busur yang menghubungkannya.
$u.\pi$	Predecessor (parent) u di dalam tree: ujung lain dari busur termurah tersebut.
Q	Antrean prioritas berisi simpul yang **belum** masuk tree, diurutkan menurut key.
$w(u, v)$	Bobot busur (u, v).

Tabel 1. Atribut dan notasi pada algoritma Prim.

Langkah	Penjelasan
1–4	Inisialisasi: key semua simpul ∞, kecuali akar r yang bernilai 0; predecessor semua simpul NIL.
5	Q berisi semua simpul graf.
6	Selama Q tidak kosong, jalankan langkah 7 dan 8. Jika Q kosong, algoritma selesai.
7	`EXTRACT-MIN(Q)` : keluarkan simpul u dengan key terkecil dari Q; u resmi masuk tree. Pada iterasi pertama, yang di-extract adalah akar.
8	Periksa setiap v dalam $Adj[u]$ sesuai urutan adjacency-list.
9	Jika v masih di Q **dan** busur (u, v) lebih ringan dari key v saat ini, lanjut ke langkah 10–11; jika tidak, periksa v berikutnya.
10–11	Perbarui: $v.\pi = u$ dan $v.key = w(u, v)$. Dengan kata lain, cara termurah menyambungkan v sekarang adalah lewat u.

Tabel 2. Penjelasan langkah-langkah algoritma Prim.

Setelah algoritma selesai, MST adalah himpunan busur $\{(v, v.\pi) : v \in V - \{r\}\}$, dan bobotnya adalah jumlah key semua simpul pada saat simpul itu di-extract.

CONTOH DAN PENERAPAN

3. Penelusuran Prim dengan Akar a

Graf contoh sama dengan Modul 19–20. Adjacency-list dari bahan kuliah:

a	b	c	d	e	f	g	h	i
b, h	a, c, h	b, i, f, d	c, e, f	d, f	c, d, e, g	f, h, i	a, b, g, i	c, g, h

Tabel 3. Adjacency-list graf contoh.

Langkah 1–4. $a.key = 0$, semua key lainnya ∞, semua predecessor NIL. **Langkah 5.** $Q = \{a, b, c, d, e, f, g, h, i\}$.

3.1 Iterasi while (langkah 6–11)

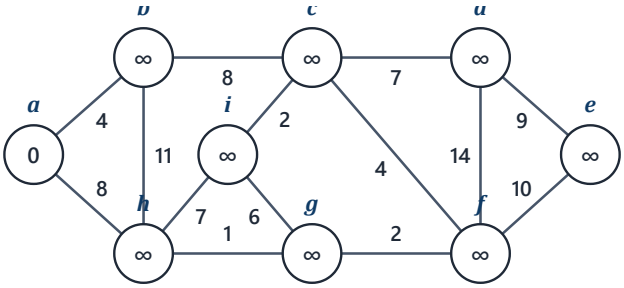
Iter.	Extract u (key)	Pembaruan (langkah 10–11)	Key simpul di Q setelah iterasi
1	a (0)	b.key = 4, b.π = a; h.key = 8, h.π = a	b 4, c ∞, d ∞, e ∞, f ∞, g ∞, h 8, i ∞
2	b (4)	c.key = 8, c.π = b (h: $w(b, h) = 11 > 8$, tidak berubah)	c 8, d ∞, e ∞, f ∞, g ∞, h 8, i ∞
3	c (8)	i.key = 2; f.key = 4; d.key = 7 (π = c)	d 7, e ∞, f 4, g ∞, h 8, i 2
4	i (2)	g.key = 6, g.π = i; h.key = 7, h.π = i	d 7, e ∞, f 4, g 6, h 7
5	f (4)	e.key = 10, e.π = f; g.key = 2, g.π = f (d: $14 > 7$)	d 7, e 10, g 2, h 7
6	g (2)	h.key = 1, h.π = g	d 7, e 10, h 1
7	h (1)	tidak ada (a, b, g, i sudah keluar dari Q)	d 7, e 10

8	d (7)	e.key = 9, e.π = d	e 9
9	e (9)	tidak ada	kosong

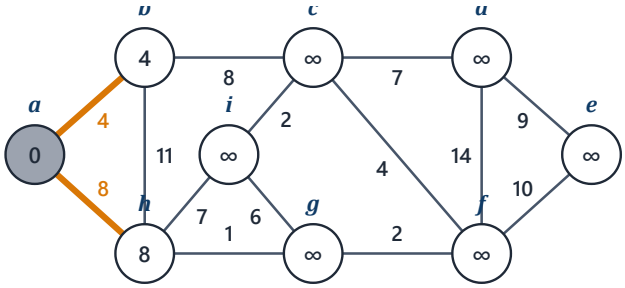
Tabel 4. Setiap iterasi Prim: simpul yang di-extract dan key yang diperbarui. Kolom terakhir menunjukkan key simpul yang masih di Q.

Pada iterasi 3, c dan h sama-sama memiliki key 8. Bahan kuliah memilih c **berdasarkan urutan simpul di himpunan** Q (urutan abjad). Pilihan lain (mengambil h lebih dulu) juga sah dan tetap menghasilkan MST berbobot 37.

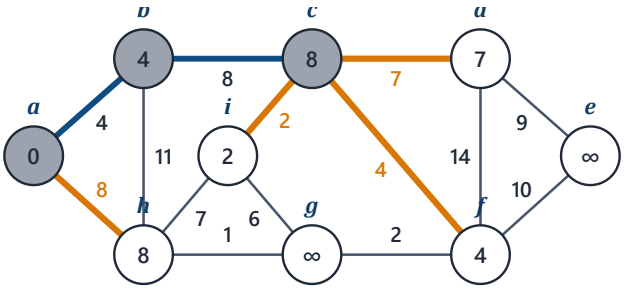
Gambar berikut memperlihatkan beberapa keadaan. Angka di dalam simpul adalah key; simpul abu-abu sudah di-extract (sudah masuk tree); busur biru tebal adalah busur tree $(v, v.\pi)$ untuk simpul yang sudah di-extract; busur oranye adalah busur $(v, v.\pi)$ sementara untuk simpul yang masih di Q (calon sambungan termurah).



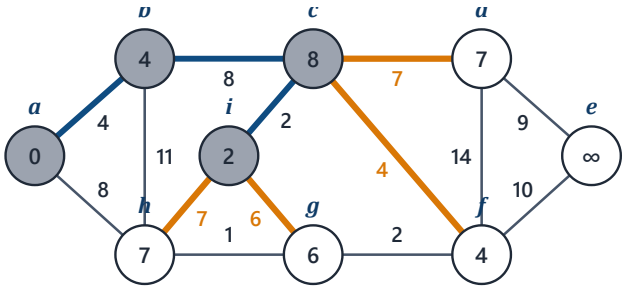
Gambar 1. (a) Langkah 1–5: semua key = ∞ kecuali akar a (key 0).



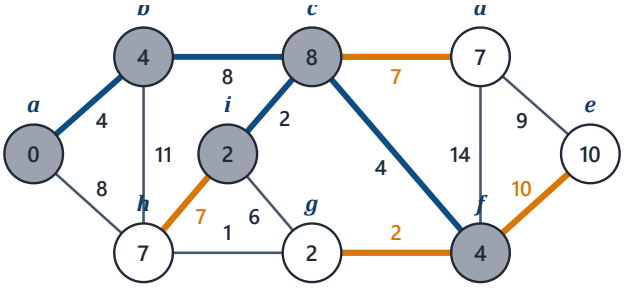
Gambar 2. (b) Setelah extract a: b.key=4, h.key=8.



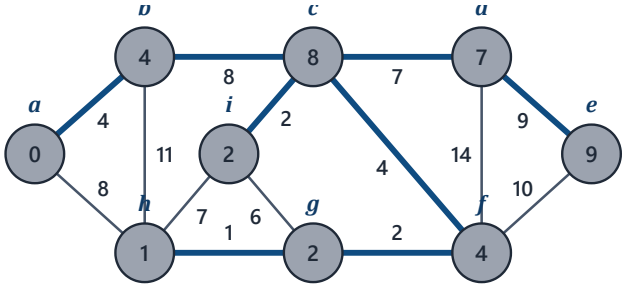
Gambar 3. (c) Setelah extract c: i.key=2, f.key=4, d.key=7.



Gambar 4. (d) Setelah extract i: g.key=6, h.key=7.



Gambar 5. (e) Setelah extract f: e.key=10, g.key=2.



Gambar 6. (f) Setelah extract e: tidak ada key yang berubah.

3.2 Hasil

Busur MST adalah $(v, v.\pi)$ untuk setiap $v \neq a$:

$$\{(b,a),(c,b),(i,c),(f,c),(g,f),(h,g),(d,c),(e,d)\},$$

dengan bobot total $4+8+2+4+2+1+7+9 = 37$, yaitu jumlah key setiap simpul saat di-extract. Hasil ini sama dengan himpunan $\{(a,b),(b,c),(c,d),(c,f),(c,i),(d,e),(f,g),(g,h)\}$ pada bahan kuliah.

3.3 Implementasi Python

Implementasi berikut memakai binary heap (`heapq`). Karena `heapq` tidak menyediakan operasi "turunkan key", setiap pembaruan key dimasukkan sebagai entri baru, dan entri lama yang sudah usang diabaikan saat dikeluarkan. Pasangan (key, nama simpul) membuat simpul dengan key sama dikeluarkan berdasarkan abjad, sesuai bahan kuliah.

Python: MST-PRIM dengan binary heap

```python
import heapq
INF = float("inf")

def prim(Adj, w, r):
    key = {u: INF for u in Adj}
    pi = {u: None for u in Adj}
    key[r] = 0
    Q = [(0, r)]
    di_tree, urutan = set(), []
    while Q:
        k, u = heapq.heappop(Q)          # EXTRACT-MIN
        if u in di_tree:
            continue                     # entri usang, lewati
        di_tree.add(u); urutan.append(f"{u}({k})")
        for v in Adj[u]:
            if v not in di_tree and w[u, v] < key[v]:
                key[v], pi[v] = w[u, v], u
                heapq.heappush(Q, (key[v], v))
    return pi, key, urutan

busur = [("a","b",4), ("a","h",8), ("b","c",8), ("b","h",11), ("c","d",7),
         ("c","i",2), ("c","f",4), ("d","e",9), ("d","f",14), ("e","f",10),
         ("f","g",2), ("g","h",1), ("g","i",6), ("h","i",7)]
w = {}
for u, v, b in busur:
    w[u, v] = w[v, u] = b
Adj = {"a": "bh", "b": "ach", "c": "bifd", "d": "cef", "e": "df",
       "f": "cdeg", "g": "fhi", "h": "abgi", "i": "cgh"}
pi, key, urutan = prim(Adj, w, "a")
print(" ".join(urutan))
print(sorted(v + pi[v] for v in Adj if pi[v]), sum(key.values()))
```

Keluaran program

```
a(0) b(4) c(8) i(2) f(4) g(2) h(1) d(7) e(9)
['ba', 'cb', 'dc', 'ed', 'fc', 'gf', 'hg', 'ic'] 37
```

ANALISIS DAN HUBUNGAN KONSEP

4. Kebenaran, Running Time, dan Perbandingan

4.1 Mengapa Prim benar?

Misalkan S adalah himpunan simpul yang sudah di-extract (sudah di tree) dan A adalah busur tree di antara mereka. Cut $(S, V - S)$ respects A, karena semua busur A berada di dalam S. Untuk setiap simpul v di luar tree, $v.key$ adalah bobot crossing edge termurah yang menyentuh v. Jadi simpul yang di-extract berikutnya, yaitu yang key-nya terkecil, terhubung lewat **light edge** cut tersebut. Menurut Teorema 23.1 (Modul 19), busur itu **safe**. Karena setiap busur yang ditambahkan aman, hasil akhirnya MST.

4.2 Running time

Running time Prim bergantung pada implementasi antrean prioritas Q (Cormen dkk., 2009, Bab 23.2):

Implementasi Q	EXTRACT-MIN ($\|V\|$ kali)	Pembaruan key (paling banyak $\|E\|$ kali)	Total
Array biasa	$O(V)$ per operasi → $O(V^2)$	$O(1)$ per operasi → $O(E)$	$O(V^2)$
Binary min-heap	$O(\lg V)$ → $O(V \lg V)$	DECREASE-KEY $O(\lg V)$ → $O(E \lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$ amortized → $O(V \lg V)$	$O(1)$ amortized → $O(E)$	$O(E + V \lg V)$

Tabel 5. Running time Prim untuk beberapa implementasi Q.

Langkah 8–11 secara total memeriksa setiap adjacency-list sekali, yaitu $\Theta(E)$ pemeriksaan untuk seluruh algoritma. Untuk graf padat ($|E| \approx |V|^2$), implementasi array $O(V^2)$ justru sudah optimal; untuk graf jarang, binary heap lebih baik. Fibonacci heap berasal dari Fredman dan Tarjan (1987).

4.3 Prim dibandingkan dengan Kruskal

Aspek	Kruskal (Modul 20)	Prim
Yang ditumbuhkan	Forest yang menyatu	Satu tree dari akar r
Yang dipilih per langkah	Busur termurah di seluruh graf yang tidak membentuk cycle	Simpul di luar tree dengan sambungan termurah
Representasi graf	Daftar busur	Adjacency-list
Struktur data	Himpunan saling lepas	Antrean prioritas minimum
Running time	$O(E \lg V)$	$O(E \lg V)$ (heap) atau $O(V^2)$ (array)
Urutan busur ditemukan pada contoh	(h,g), (c,i), (g,f), (a,b), ...	(a,b), (b,c), (c,i), (c,f), ...

Tabel 6. Perbandingan Prim dan Kruskal.

Kemiripan dengan Dijkstra. Pseudocode Prim hampir identik dengan algoritma Dijkstra (Modul 24). Perbedaannya hanya pada makna key: pada Prim, key adalah bobot **satu busur** yang menyambungkan simpul ke tree ($v.key = w(u, v)$); pada Dijkstra, $v.d$ adalah **total bobot lintasan** dari sumber ($v.d = u.d +$

$w(u, v)$). Karena itu hasilnya berbeda: Prim menghasilkan MST, Dijkstra menghasilkan pohon lintasan terpendek.

KESIMPULAN

5. Kesimpulan

- Prim menumbuhkan satu tree dari akar r: setiap iterasi meng-extract simpul dengan key terkecil, lalu memperbarui key tetangganya yang masih di Q.
- $v.key$ adalah bobot busur termurah yang menyambungkan v ke tree; $v.\pi$ adalah ujung lain busur tersebut.
- Pada graf contoh dengan akar a, urutan extract: a, b, c, i, f, g, h, d, e; MST berbobot 37, sama dengan bahan kuliah.
- Setiap simpul yang di-extract disambungkan lewat light edge cut antara tree dan sisa graf, sehingga Prim benar.
- Running time $O(E \lg V)$ dengan binary heap, $O(V^2)$ dengan array, $O(E + V \lg V)$ dengan Fibonacci heap.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Pada Prim, nilai $v.key$ untuk simpul v yang masih di Q menyatakan ...

A. Jarak terpendek dari akar ke v.
B. Bobot busur termurah yang menghubungkan v dengan tree saat ini.
C. Banyaknya tetangga v.
D. Urutan v di adjacency-list.

2 PENELUSURAN

Jalankan Prim dari akar p pada graf dengan busur $(p,q)=3$, $(p,r)=1$, $(q,r)=2$, $(q,s)=5$, $(r,s)=4$, $(s,t)=6$, $(r,t)=7$. Tuliskan urutan extract beserta key-nya dan total bobot MST.

3 ANALISIS

Pada iterasi 4 penelusuran di modul ini, key h turun dari 8 menjadi 7, lalu pada iterasi 6 turun lagi menjadi 1. Jelaskan apa arti setiap perubahan itu.

4 BENAR/SALAH DENGAN ALASAN

"Jika Prim dijalankan dari akar yang berbeda, bobot MST yang dihasilkan bisa berbeda." Benar atau salah?

5 PERHITUNGAN

Sebuah graf padat memiliki $|V| = 1.000$ dan $|E| \approx 500.000$. Bandingkan perkiraan running time Prim dengan array (V^2) dan dengan binary heap ($E \lg V$). Mana yang lebih baik?

6 ANALISIS KASUS

Jalankan Prim pada graf contoh dengan akar e (gunakan aturan abjad bila key sama). Simpul mana yang di-extract pertama kali setelah e, dan berapa bobot MST-nya?

7 PILIHAN GANDA

Mengapa pada langkah 9 diperiksa syarat "$v \in Q$"?

A. Agar simpul yang sudah masuk tree tidak diperbarui lagi; jika diperbarui, struktur tree bisa rusak.
B. Agar running time menjadi $O(V)$.
C. Agar busur berbobot negatif diabaikan.
D. Karena Q selalu kosong setelah iterasi pertama.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. Pilihan A adalah arti $v.d$ pada Dijkstra, bukan key pada Prim.

Soal 2

Extract p (0): $q.key = 3$, $r.key = 1$. Extract r (1): $q.key = 2$, $s.key = 4$, $t.key = 7$. Extract q (2): $w(q, s) = 5 > 4$, tidak berubah. Extract s (4): $t.key = 6$. Extract t (6). Urutan: **p(0), r(1), q(2), s(4), t(6)**; MST $\{(r, p), (q, r), (s, r), (t, s)\}$, total **13**, sama dengan hasil Kruskal pada kuis Modul 20.

Soal 3

Awalnya ($h.key = 8$, $\pi = a$) cara termurah menyambungkan h adalah lewat busur (a, h). Setelah i masuk tree, ditemukan sambungan lebih murah $(i, h) = 7$. Setelah g masuk tree, ditemukan sambungan yang lebih murah lagi $(g, h) = 1$. Key selalu mencerminkan sambungan termurah yang diketahui sejauh ini ke tree yang terus membesar.

Soal 4

Salah. Akar yang berbeda dapat menghasilkan urutan extract dan bahkan MST yang berbeda (bila ada bobot sama), tetapi **bobot** MST selalu sama, yaitu bobot minimum. Pada graf contoh, bobotnya selalu 37.

Soal 5

Array: $V^2 = 10^6$ langkah. Binary heap: $E \lg V \approx 500.000 \times 10 = 5 \times 10^6$ langkah. Untuk graf padat ini, **array lebih baik** (sekitar 5 kali lebih sedikit).

Soal 6

Dari e: $Adj[e] = d, f$ memberi $d.key = 9$, $f.key = 10$, sehingga yang di-extract pertama setelah e adalah d (key 9). Selanjutnya c (7), i (2), f (4), g (2), h (1), a (8, lewat (a, h) karena h sudah di tree sebelum b), b (4). Bobot MST tetap **37** ($9 + 7 + 2 + 4 + 2 + 1 + 8 + 4$). MST yang terbentuk kali ini memuat (a, h), yaitu T_2 dari Modul 19.

Soal 7

Jawaban: A. Simpul yang sudah di-extract sudah memiliki busur tree yang final. Memperbarui π-nya dapat membentuk cycle atau memutus tree.

Referensi

Sumber utama: Materi 12. MINIMUM SPANNING TREE, algoritma Kruskal dan Prim (bagian Algoritma Prim) (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Prim, R. C. (1957). Shortest Connection Networks and Some Generalizations. *Bell System Technical Journal*, 36(6), 1389–1401. https://doi.org/10.1002/j.1538-7305.1957.tb01515.x

3. Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms. *Journal of the ACM*, 34(3), 596–615. https://doi.org/10.1145/28869.28874
4. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.